

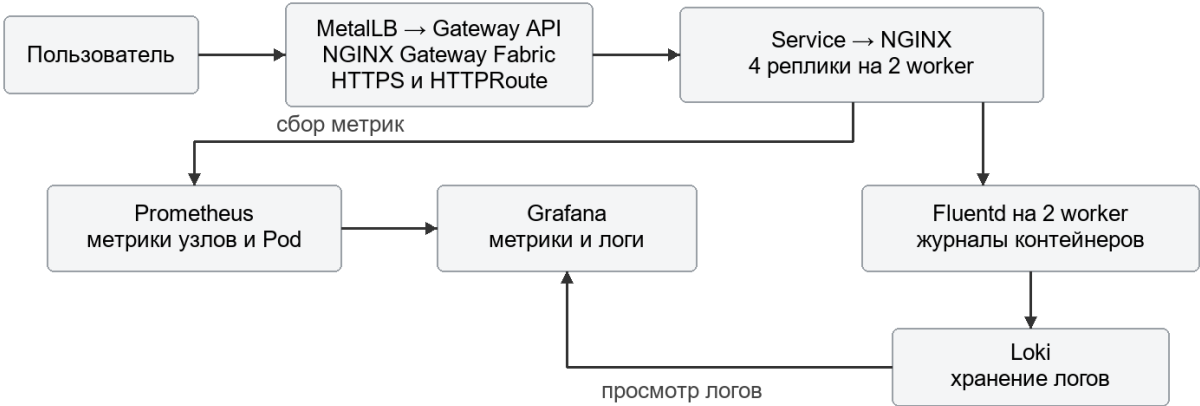
Паспорт проекта Kubernetes

Архитектура и состав решения

Решение хакатона разворачивает NGINX-приложение в трёхузловом Kubernetes-кластере на Ubuntu 24.04. Внешний доступ реализован через Gateway API и HTTPS; Prometheus собирает метрики, Fluentd передаёт логи в Loki, Grafana предоставляет интерфейс наблюдения.

Компонент	Технологии и конфигурация
Среда и кластер	Ubuntu 24.04; Kubernetes v1.35.9; kubeadm; containerd; 1 control-plane + 2 worker
Сеть и внешний доступ	Calico v3.33.0; Pod CIDR 10.244.0.0/16; MetalLB v0.16.1
Gateway API и TLS	NGINX Gateway Fabric 2.7.2; Gateway и HTTPRoute; cert-manager v1.21.2
Приложение	NGINX, 4 реплики; Deployment и Service; распределение по worker-нодам
Наблюдаемость	Helm chart kube-prometheus-stack 91.9.0; Prometheus и Grafana; Fluentd → Loki → Grafana
Автоматизация	Shell-скрипты, kubectl apply, Helm; deploy.sh и verify.sh; настройки в cluster.env

Схема взаимодействия



Кластер kubeadm 1 control-plane + 2 worker | сеть Pod 10.244.0.0/16

Параметры LAN, пул MetalLB и hostname задаются в cluster.env. Сеть Pod 10.244.0.0/16 отделена от LAN 192.168.3.0/24; Gateway связывает внешний запрос с Service приложения. Порядок запуска: подготовка трёх узлов → инициализация control-plane → подключение worker → ./deploy.sh → ./verify.sh. Подробные команды и конфигурация размещаются в README и исходных файлах репозитория.

Реализованный обязательный функционал

Воспроизводимое развёртывание Kubernetes

Реализация и выбор. `prepare-node.sh` готовит узлы Ubuntu 24.04, `init-control-plane.sh` создаёт кластер `kubeadm` и устанавливает `Calico`, `join-worker.sh` подключает два `worker`. Конфигурация стенда вынесена в `cluster.env`; `kubeadm` позволяет воспроизвести кластер на собственных машинах.

Проверка. Выполнить скрипты по README, затем `./deploy.sh` и `./verify.sh`. Команда `kubectl get nodes -o wide` должна показать три узла Ready и Kubernetes v1.35.9.

Приложение и маршрутизация через Gateway API

Реализация и выбор. NGINX развёрнут как Deployment с четырьмя репликами и Service. NGINX Gateway Fabric обслуживает Gateway и HTTPRoute; MetalLB выдаёт внешний IP. Gateway API задаёт входную точку и правила маршрутизации отдельными Kubernetes-ресурсами.

Проверка. `kubectl get pods -n hackathon -o wide`: четыре Pod приложения Ready. Проверить Gateway Programmed=True, HTTPRoute Accepted=True и ResolvedRefs=True; HTTPS-запрос должен вернуть страницу Hello World!.

Сбор метрик в Prometheus

Реализация и выбор. `kube-prometheus-stack` установлен через Helm и объединяет Prometheus, Grafana и экспортеры метрик. Готовый стек сокращает ручную настройку и позволяет наблюдать состояние узлов и Kubernetes-ресурсов.

Проверка. В Prometheus открыть Targets и выполнить PromQL `up, kube_node_info` и `kube_pod_info{namespace="hackathon"}`. Подтверждение сбора: рабочий target со значением `up=1` и непустые актуальные серии метрик.

Сбор и просмотр логов через Fluentd

Реализация и выбор. Fluentd работает как DaemonSet на worker-нодах, читает контейнерные журналы NGINX и отправляет их в Loki. Grafana предоставляет единый интерфейс поиска логов и просмотра метрик.

Проверка. `kubectl get ds fluentd -n logging`: две готовые копии. Отправить пять HTTPS-запросов к приложению; в Grafana Explore выбрать Loki и запрос `{application="nginx"} |= "GET"`. Ожидаются новые записи в выбранном интервале времени.

Исходные файлы и инструкция воспроизведения

Реализация и выбор. Манифесты, настройки Helm и скрипты сопровождаются README с версиями, требованиями к среде, порядком запуска, проверками и ограничениями. `cluster.env` создаётся из примера; `join.env` содержит временные данные подключения worker.

Проверка. Клонировать публичную ветку `main` без авторизации и выполнить README. Убедиться, что `cluster.env` и `join.env` не отслеживаются Git, а реальные пароли, токены и приватные ключи отсутствуют в репозитории.

Дополнительные улучшения и проверка

TLS на входе в приложение

Реализация и обоснование. cert-manager выпускает self-signed сертификат, а HTTPS listener Gateway завершает TLS. Это даёт шифрование трафика в локальном стенде без внешнего DNS и публичного удостоверяющего центра.

Проверка. `kubectl get certificate -n hackathon: Ready=True`; запрос по HTTPS возвращает приложение. `curl -k` допускает self-signed сертификат, но не проверяет доверие к его издателю.

Несколько узлов и четыре реплики

Реализация и обоснование. Два worker-узла и topologySpreadConstraints распределяют реплики NGINX по узлам. Такая схема уменьшает зависимость приложения от одного worker и показывает работу планировщика в multi-node кластере.

Проверка. `kubectl get pods -n hackathon -o wide`: четыре готовые реплики размещены на двух worker. После удаления одного Pod контроллер восстанавливает количество реплик до четырёх.

LoadBalancer в локальной сети

Реализация и обоснование. MetalLB использует IPAddressPool и L2Advertisement. Gateway получает адрес из свободного диапазона LAN без облачного балансировщика; диапазон задаётся параметрами cluster.env.

Проверка. `kubectl get ipaddresspool,l2advertisement -n metallb-system`; адрес Gateway входит в пул и доступен с машины в LAN. Адреса пула должны быть исключены из DHCP и не заняты другими устройствами.

Централизованные логи двух worker

Реализация и обоснование. Fluentd собирает журналы с обоих worker, Loki хранит их, Grafana выполняет запросы по метке `application="nginx"`. Для диагностики не требуется вручную открывать каждый узел.

Проверка. `Fluentd Ready=2`; после обращений к приложению появляются новые access-логи в Loki. Метрики и логи доступны через источники данных Grafana.

Повторяемое развёртывание

Реализация и обоснование. `deploy.sh` применяет манифесты через `kubectl apply` и устанавливает Helm-компоненты через `helm upgrade --install`. Повторный запуск обновляет существующие ресурсы и упрощает восстановление конфигурации.

Проверка. Повторить `./deploy.sh`, затем `./verify.sh`; убедиться, что не появились дубли Helm-релизов и приложение продолжает отвечать. Идемпотентность относится к развёртыванию компонентов в подготовленном кластере.

Автоматическая проверка стенда

Реализация и обоснование. `verify.sh` проверяет узлы, приложение, Gateway API, TLS, monitoring, Fluentd и Loki. IP Gateway определяется из Kubernetes, hostname берётся из cluster.env; проверка переносится между стендами.

Проверка. `./verify.sh` завершается без FAIL; отдельно подтвердить реальный сбор метрик запросом PromQL и появление новых логов запросом LogQL. Готовность Pod сама по себе не доказывает поступление данных.

Ревью и дальнейшее развитие

Сильная техническая сторона

Решение объединяет воспроизводимое развёртывание трёхузлового кластера, внешний доступ через Gateway API и наблюдаемость приложения. Скрипты подготовки, `deploy.sh` и `verify.sh` дают эксперту последовательный путь от чистых Ubuntu 24.04 до работающего HTTPS-приложения с метриками и логами.

Самое сложное решение

Исходный Pod CIDR 192.168.0.0/16 пересекался с LAN 192.168.3.0/24, создавая конфликт маршрутизации между сетью Pod и физическими узлами. Выбор был между изменением адресации LAN и переносом сети Pod; выбрана сеть 10.244.0.0/16, чтобы сохранить адреса узлов и устранить пересечение. Новый CIDR согласован в настройках `kubeadm` и `Calico`; результат проверяется готовностью трёх узлов, сетевой связностью Pod и доступом через Gateway.

Ревью и границы текущего решения

Стенд демонстрирует работу обязательных компонентов, но одна control-plane нода остаётся единой точкой отказа управления. Self-signed TLS требует доверия к локальному сертификату; Loki работает в одном экземпляре, а административный доступ к Prometheus и Grafana выполняется через port-forward. Распределение четырёх реплик по двум worker не заменяет полноценную отказоустойчивость control-plane и хранилищ.

Развитие для телекоммуникаций

- Отказоустойчивость и резервное копирование. Три control-plane, балансировщик Kubernetes API, резервные копии etcd и тест восстановления; нужны дополнительные узлы и хранилище резервных копий.
- Долговременное хранение телеметрии. PersistentVolume, отказоустойчивый Loki и объектное хранилище, политика retention; нужны CSI-драйвер, ёмкость дисков и S3-совместимое хранилище.
- Безопасность операторской инфраструктуры. NetworkPolicy, RBAC, корпоративный CA и управление секретами; нужны модель доступа, PKI и согласованный перечень сетевых взаимодействий.
- Телеком-нагрузки и несколько сетей. Multus и SR-IOV для выделенных сетевых интерфейсов CNF, CPU pinning и hugepages; нужны поддерживаемые NIC, настройка хостов и нагрузочные испытания.
- Эксплуатация по SLO и масштабирование. Алерты по доступности, задержке и ошибкам, HPA, испытания отказов и GitOps; нужны измеримые SLO, нагрузочный генератор и ресурсы для масштабирования.